SKRIBBL.IO CLONE

Full-Stack System Architecture, Real-Time Protocol & Database Design Specification

Role: Senior Full-Stack Architect & Coach	**Technology:** React, Node.js, Socket.IO, MySQL, Docker
Target: Final Technical Interview Round	**Document Status:** Phase 0 Approved Baseline

1. Project Overview & Multiplayer Walkthrough

The project is an interactive, full-stack multiplayer clone of the beloved web game **skribbl.io**. Players gather in virtual rooms where they take turns drawing secret words on an HTML5 canvas while other players race to guess the word in a real-time chat feed. The backend acts as the authoritative arbitrator of game state, managing turns, timers, word masking, score computation, and broadcast routing.

Core Application Screens & User Flow:

Screen / View	User Purpose	Key Technical Interactions
Create Room View	Host enters nickname, sets max players (2-20), rounds (2-10), draw time (15-240s), hints count.	Emits create_room; backend persists room record in MySQL & allocates in-memory Room.
Lobby View	Participants view joined roster, avatars, host badge, and invite link. Host clicks 'Start Game'.	Listens to player_joined, player_left. Only host socket can emit start_game.
Active Game View	Drawer selects word and draws; guessers view live strokes, masked blanks, and submit guesses in chat.	Streams normalized vector strokes via draw_data; backend authoritatively runs timer & scores.
Round End View	Reveals secret word, awards points to solvers and drawer, displays updated rankings.	Server flushes round scores to MySQL; triggers next drawer or initiates game conclusion.
Podium / Game Over	Celebrates 1st, 2nd, and 3rd place winners with final match stats and 'Play Again' button.	Marks game_sessions.status = 'COMPLETED'; archives match records in database.

Multiplayer Example Walkthrough (Alice, Bob, Charlie, Dana):
Alice creates room SKR-482 with 60s draw time. Bob, Charlie, and Dana join via link. Alice clicks 'Start Game'. Alice is designated Drawer for Round 1. The server generates 3 choices: *['Rocket', 'Castle', 'Guitar']* and emits them **only to Alice**. Bob, Charlie, and Dana receive masked blanks: _ _ _ _ _ _ (6 letters). Alice chooses 'Rocket'. When Alice draws, coordinates are streamed across Socket.IO. Bob guesses 'Airplane' (fails, visible in chat). Charlie guesses 'Rocket' (correct!). Charlie earns 450 points; Alice receives 100 drawer points. Bob and Dana see: *'Charlie guessed the word!'* without the word being leaked. When time expires, scores persist to MySQL, and Bob becomes the next drawer.

2. Technology Stack & Comparison with Java / Spring Boot

As a developer with a solid foundation in Java, Spring Boot, OOP, and relational SQL, understanding how the chosen JavaScript/TypeScript stack maps to familiar enterprise patterns is essential for confident interview explanations.

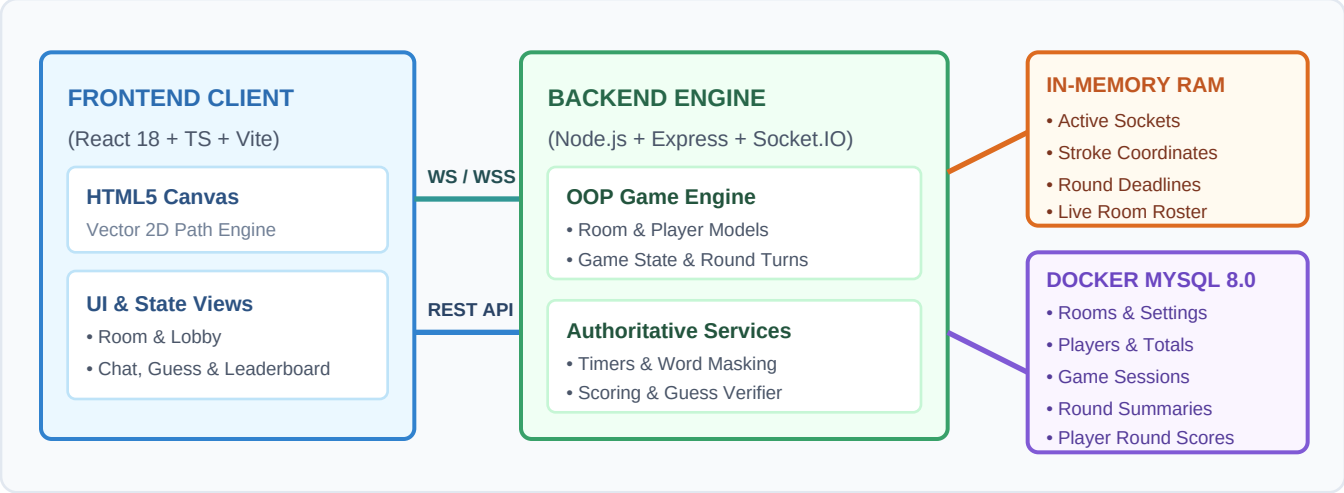
Layer	Chosen Technology	Architectural Purpose	Java / Spring Boot Equivalent
Frontend SPA	React 18 + TS + Vite	Reactive component UI, type-safety, instant HMR	React / Thymeleaf frontend integration
Canvas Layer	HTML5 Canvas 2D API	Direct pixel manipulation, zero library overhead	JavaFX / Swing Graphics2D drawing
Backend Server	Node.js + Express.js	Event-loop I/O, REST endpoints, static asset server	Spring Boot MVC (@RestController)
Real-Time Engine	Socket.IO v4	Bi-directional event pub/sub with WebSocket transport	Spring WebSocket + STOMP (SimpMessagingTemplate)
Relational DB	MySQL 8.0 (Docker)	ACID transactions, structured audit of matches & scores	MySQL / PostgreSQL with Spring Data JPA & Hibernate
DB Connectivity	mysql2 Connection Pool	Prepared statements, parameterized queries, pooling	HikariCP connection pool + JdbcTemplate / JPA
Live Game State	RAM In-Memory Map	Sub-millisecond access for strokes, timers, sockets	ConcurrentHashMap / Redis caching beans

Key Architectural Synergies:

1. Event-Driven Concurrency: Node.js executes JavaScript on a single thread with an event-driven non-blocking I/O loop. Unlike standard thread-per-request servlet models in traditional Java servers, Node.js handles thousands of concurrent WebSocket connections with minimal memory footprint.
2. OOP Domain Modeling in TypeScript: To maintain enterprise design quality, backend business logic is encapsulated in OOP classes (Room, Game, Round, Player), perfectly mirroring Spring Boot domain-driven architecture.
3. Authoritative Game Rules: Just like a Spring Boot secure service layer, all critical game state transitions (word validation, round timers, drawer rotation, score calculation) are exclusively executed on the server, ensuring clients cannot manipulate gameplay.

3. High-Level System Architecture

The system architecture strictly decouples **real-time ephemeral gameplay** from **persistent historical storage**. The React frontend communicates over persistent WebSockets (Socket.IO) for high-frequency gameplay events and standard HTTP REST for stateless operations like health checks.

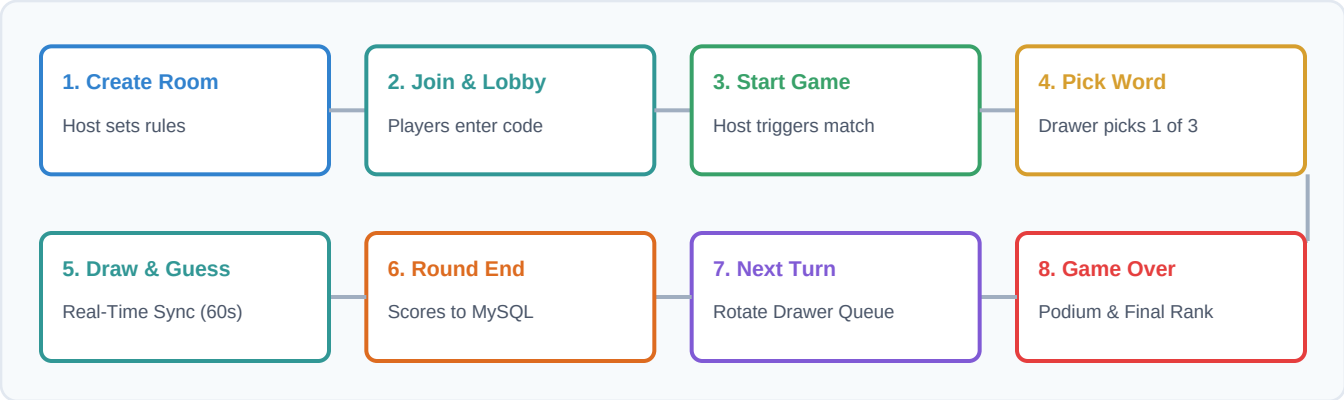


Component Responsibilities Breakdown:

Component	Runtime Location	Responsibilities & Guarantees
React UI + Canvas	Browser (Client)	Captures mouse/pointer events, converts to normalized (0..1) coordinates, renders received vector stroke sequences, displays masked secret words and chat.
Socket.IO Gateway	Node.js (Server)	Maintains full-duplex socket connections, maps socket.id to player_id, handles room join/leave broadcasts, filters unauthorized incoming events.
OOP Game Engine	Node.js (Server)	Authoritative state machine (Room, Game, Round). Controls turn rotation, picks random words, runs authoritative 60s countdown clock, validates word guesses.
In-Memory Store	Server RAM	Holds stroke buffer for mid-round joiners and undo operations, secret word strings, and active timer intervals. Zero disk I/O during drawing.
MySQL 8.0 Container	Docker Volume	Permanently archives room configs, game sessions, completed round summaries, player leaderboards, and categorized word dictionary.

4. Complete Game Workflow Flowchart

The complete end-to-end game lifecycle progresses through strict, server-controlled state machine phases. At no point can a client trigger a state transition without server validation.



Phase-by-Phase Technical Progression:

1. Room Creation: Host submits configuration form. Server creates row in MySQL rooms table with status 'LOBBY' and allocates an in-memory Room instance.
2. Joining & Roster Sync: Players join by entering the room code. Sockets join the Socket.IO room channel. Server emits player_joined to update all client lobby screens in real time.
3. Game Initiation: Host clicks 'Start Game'. Server verifies sender socket matches hostPlayerId, updates rooms.status = 'PLAYING', initializes Game, and queues player turns.
4. Word Selection Phase: Server picks 3 random words from the word catalog. Emits round_start with word options **exclusively to the active drawer**. Guessers receive masked blank format.
5. Live Drawing & Guessing: Drawer selects word; server starts authoritative countdown timer. Drawer emits stroke packets. Guessers submit guesses in chat. Correct guessers earn speed-based points.
6. Turn Concession / Round End: When time expires or all players guess correctly, server reveals secret word, calculates drawer compensation points, flushes scores to MySQL, and rotates to next drawer.
7. Game Conclusion: When all configured rounds conclude, server calculates final podium standings, updates MySQL game_sessions.status = 'COMPLETED', and emits game_over.

5. Relational Database Design & Schema Specifications

The relational schema is designed in 3rd Normal Form (3NF) to provide relational integrity, eliminate redundancy, and support historical analytics (e.g. win rates, top words, average guess times). It runs inside Docker with persistent storage.

Table: rooms (Master Room Configuration & Status)

Column	Type	Constraints	Description & Purpose
`room_id`	VARCHAR(10)	PRIMARY KEY	Unique alphanumeric room code (e.g. 'SKR-482')
`room_name`	VARCHAR(64)	NOT NULL	Display name for public/private room identification
`host_name`	VARCHAR(64)	NOT NULL	Screen name of the room creator
`max_players`	INT	DEFAULT 8	Maximum concurrent participants allowed (2-20)
`total_rounds`	INT	DEFAULT 3	Number of complete rounds to play (2-10)
`draw_time_seconds`	INT	DEFAULT 80	Duration of each drawing turn (15-240s)
`is_private`	BOOLEAN	DEFAULT FALSE	True if invite-only; false if visible to public matchmaker
`status`	ENUM(...)	DEFAULT 'LOBBY'	'LOBBY', 'PLAYING', 'FINISHED', 'ABANDONED'
`created_at`	TIMESTAMP	CURRENT_TIMESTAMP	Creation timestamp for audit and TTL cleanup

Table: players (Participant Profiles)

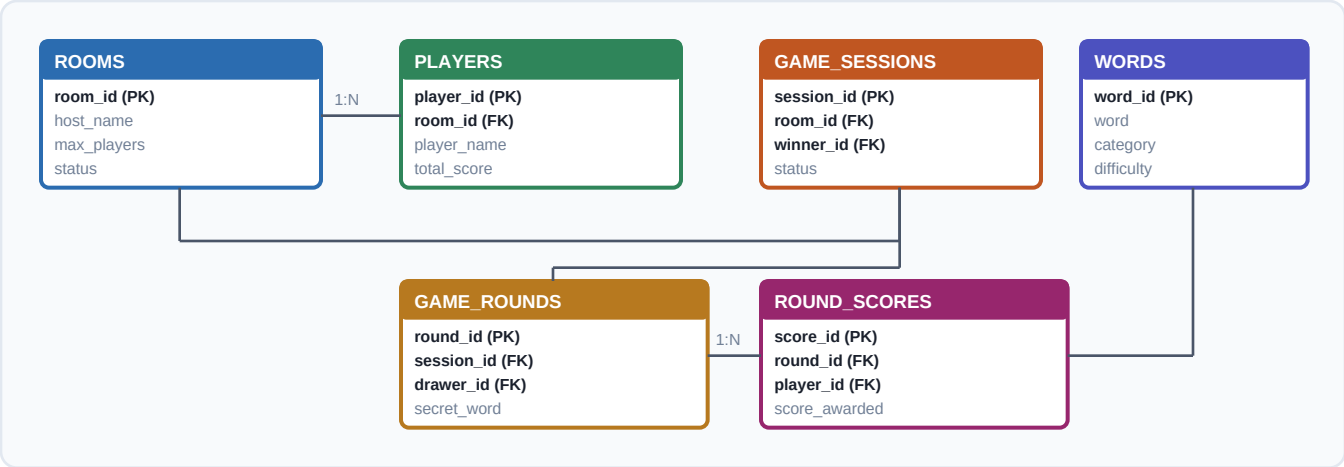
Column	Type	Constraints	Description & Purpose
`player_id`	VARCHAR(36)	PRIMARY KEY	UUID v4 identifying the player session
`room_id`	VARCHAR(10)	FK -> rooms(room_id) ON DELETE CASCADE	Associated room identifier
`player_name`	VARCHAR(64)	NOT NULL	Display nickname selected by user
`avatar_id`	VARCHAR(32)	DEFAULT 'avatar_1'	Selected avatar identifier
`is_host`	BOOLEAN	DEFAULT FALSE	True if player possesses host moderation rights
`total_score`	INT	DEFAULT 0	Cumulative score across all turns in this room
`is_connected`	BOOLEAN	DEFAULT TRUE	Real-time presence flag for disconnect detection

Tables: game_sessions, game_rounds, player_round_scores & words

Table Name	Primary Key	Key Foreign Keys	Core Columns & Functional Purpose
`game_sessions`	session_id (UUID)	room_id, winner_id	Tracks full match duration, total rounds completed, status (IN_PROGRESS, COMPLETED, ABORTED), and final winner.
`game_rounds`	round_id (UUID)	session_id, drawer_id, word_id	Records each turn: round number, turn sequence, drawer player, chosen secret word, and start/finish timestamps.
`player_round_scores`	score_id (UUID)	round_id, player_id	Detailed score audit: whether guessed correctly, seconds taken to guess, points earned, and drawer bonus points.
`words`	word_id (INT)	None (Master Catalog)	Word dictionary with categories (ANIMALS, OBJECTS, ACTIONS, FOOD, PLACES) and difficulty levels (EASY, MEDIUM, HARD).

6. Visual Entity-Relationship (ER) Diagram

The following diagram illustrates the relational associations, primary keys, and foreign key constraints between the database tables. Cascading deletes ensure that removing an inactive room cleanly purges child players, sessions, and rounds without orphan records.



7. Socket.IO Real-Time Protocol Specification

WebSocket events follow a strict, typed protocol matching the assignment specification. Payloads are validated server-side to guarantee integrity and prevent payload tampering.

Event Name	Direction	Payload Structure	Functional Description
`create_room`	Client -> Server	{ hostName, settings }	Host initializes new room with custom rules
`join_room`	Client -> Server	{ roomId, playerName }	Player joins existing room by code or invite link
`player_joined`	Server -> Clients	{ player, players }	Broadcasts updated participant roster to all members
`start_game`	Client -> Server	{} (Host only)	Host triggers game start; server validates permission
`round_start`	Server -> Clients	{ drawerId, wordOptions, drawTime }	Drawer receives 3 word choices; guessers see masked blanks
`word_chosen`	Client -> Server	{ word }	Drawer selects word; server starts authoritative timer
`draw_start`	Client -> Server	{ x, y, color, size, tool }	Drawer begins stroke; verified against active drawer socket
`draw_data`	Server -> Clients	{ ...strokeCoords }	Server broadcasts stroke data to all viewers in room
`guess`	Client -> Server	{ text }	Guesser types guess; server normalizes & validates
`guess_result`	Server -> Clients	{ correct, playerId, points }	Emits points to guesser; announces solver to room
`round_end`	Server -> Clients	{ word, scores, nextDrawer }	Reveals secret word; broadcasts round score breakdown
`game_over`	Server -> Clients	{ winner, leaderboard }	All rounds complete; displays final victory podium

8. In-Memory State vs MySQL Persistence & OOP Engine

A critical engineering requirement is knowing when to write to MySQL versus when to retain data in memory. Writing every mouse coordinate or 1-second timer tick to disk would cripple database performance.

State Dimension	In-Memory Server State (RAM)	Persistent MySQL State (InnoDB)
Stored Entities	Active sockets, stroke buffer, active timers, unmasked secret word, correct solvers set, current turn queue index.	Room configs, player records, game session metadata, round summaries, player round score audit, word dictionary.
Update Frequency	30 to 60 events/second during active drawing; 1 tick/second for countdown timer.	Written strictly on milestone events: room creation, round completion, and game finish.
Performance Goal	Sub-millisecond access and instant WebSocket broadcast to prevent lag.	Permanent audit trail, historical queries, and leaderboard persistence across server restarts.

Object-Oriented Domain Model (Bonus Architecture):

Instead of unstructured procedural socket handlers, the backend server encapsulates domain logic into clean OOP classes:

- **Player:** Encapsulates socket identity, screen name, avatar, connection state, and total score.
- **Room:** Manages room lifecycle, player roster, host privileges, room status machine, and room-scoped broadcasts.
- **Game:** Controls the match state, turn queue, round progression, and podium calculation.
- **Round:** Encapsulates active drawer, secret word, word options, letter hint schedule, authoritative timer, and stroke buffer.
- **ScoringEngine:** Encapsulates point algorithms: points = Math.round(500 * (remainingTime / totalTime)).

9. Proposed Codebase Architecture

The project follows a clean, modular full-stack structure with clear separation of concerns:

Directory / Package	Role & Responsibilities
`frontend/src/components/`	Reusable UI components: Canvas, CanvasToolbar, ChatBox, Leaderboard, TimerDisplay, WordHint.
`frontend/src/pages/`	Top-level views: CreateRoom, JoinRoom, Lobby, GameView, GameOver.
`frontend/src/hooks/`	Custom React hooks: useCanvas (drawing logic), useSocket (WebSocket lifecycle and listeners).
`backend/src/game/`	OOP domain entities: Room.ts, Game.ts, Round.ts, Player.ts, ScoringEngine.ts, WordManager.ts.
`backend/src/sockets/`	Typed Socket.IO handlers: roomHandler.ts, drawingHandler.ts, chatHandler.ts, gameHandler.ts.
`backend/src/db/`	Database connection pool, schema migration scripts, and DAO query repositories.
`docker-compose.yml`	Container orchestrator running MySQL 8.0 with persistent data volume mysql_data.

10. Deployment Architecture

The application is architected for seamless local development via Docker Compose and high-performance cloud deployment. Because real-time drawing requires persistent WebSocket connections, standard serverless environments (like AWS Lambda or standard Vercel serverless functions) are paired with a persistent containerized backend.

Environment	Component	Hosting / Runtime	Configuration & Networking
Local Development	MySQL Database	Docker Compose (MySQL 8.0)	Port 3306:3306, volume mysql_data, env config via .env
Local Development	Node.js Backend	Node.js 22 + tsx (Hot Reload)	Port 5000, CORS allowed for http://localhost:5173
Local Development	React Frontend	Vite Dev Server (HMR)	Port 5173, connects to backend via ws://localhost:5000
Production Cloud	Persistent Backend	Render / Railway Web Service	Full WebSocket / WSS support, automatic health checks, zero sleep timeout
Production Cloud	Cloud Database	Railway MySQL / Aiven Cloud	Managed MySQL instance, encrypted SSL connections, persistent volume
Production Cloud	Frontend SPA	Render Static / Vercel SPA	CDN distribution, environment variable pointing to live backend WSS URL

11. Technical Interview Preparation (Questions & Answers)

Here are 5 core architecture and database questions likely to be asked in the final interview, accompanied by concise, speakable answers:

Q1: How do you prevent cheating in a multiplayer game where the client renders the canvas?

Answer: The server is the single source of truth. The secret word is strictly held in server memory and is never transmitted to guessing clients until round completion. Guessers only receive masked blanks (e.g. '_ _ _ _ _'). Additionally, only the designated drawer's socket is authorized to broadcast drawing strokes; any stroke payload emitted by a guesser is rejected on the server.

Q2: Why did you separate in-memory game state from MySQL persistence instead of writing everything to the database?

Answer: Drawing generates 30 to 60 coordinate events per second per drawer. Writing these high-frequency telemetry events directly to MySQL would saturate disk I/O and introduce noticeable lag for players. Instead, live strokes, active timers, and guess evaluation are handled entirely in Node.js RAM for sub-millisecond responsiveness. MySQL is used exclusively for structured milestone persistence: room creation, round summaries, and game results.

Q3: How do you handle coordinate scaling when different players have different monitor sizes?

Answer: We normalize all stroke coordinates on the client before transmission. Instead of sending absolute pixel coordinates like (420, 310), we transmit normalized floating-point ratios between 0.0 and 1.0 (e.g., x / canvasWidth, y / canvasHeight). When receiving clients render the stroke, they multiply these normalized ratios by their local canvas dimensions, guaranteeing identical visual drawings regardless of device resolution or aspect ratio.

Q4: How does your Node.js backend architecture compare to an enterprise Spring Boot application?

Answer: In Spring Boot, we would use Spring MVC controllers, Spring Data JPA repositories, and Spring WebSocket with STOMP messaging templates. In our Node.js stack, Express handles HTTP routes, mysql2 connection pooling handles database queries, and Socket.IO manages WebSockets. To preserve enterprise design standards, we structured the game engine using OOP principles (Room, Game, Round, and Player classes), mirroring Spring's domain-driven separation of concerns.

Q5: What happens if the room host or the active drawer abruptly disconnects?

Answer: The server listens for socket disconnect events. If the active drawer disconnects, the authoritative timer is cleared, the current round is finalized with a notification to the room, and the turn queue automatically rotates to the next player. If the room host disconnects during the lobby, the server automatically migrates host privileges to the next oldest connected player in the room.